

Nullable Reference Types in C# 8 with Examples

In this article, I am going to discuss **Nullable Reference Types in C# 8** with Examples. Please read our previous article where we discussed [Disposable Ref Structs in C#](#) with Examples.

Nullable Reference Types in C#:

Before C# 8.0, all reference types were nullable but with C# 8.0, now we can specify whether the reference is going to be null or not null. You can use the question mark “?” operator to specify a reference type as nullable type and in that case, you will not get any warning. But if you create a reference variable, and if you don't initialize that reference variable, then the compiler will give a warning when you are accessing any member of that reference variable.

C# 8.0 allows us to specify whether a variable should be null, and when it should not be null. Based on these annotations, the compiler will warn you when you are potentially using a null reference or passing a null reference to a function that will not accept it. If this is not clear at the moment, then don't worry, we will explain this with multiple examples.

In C#, whenever we create an instance of a reference type, then the object is going to be created on the heap memory. When the reference variable does not point to any object, then its value is null. Prior to C# 8.0, all reference types were nullable. With Nullable Reference Types, we might get `NullReferenceException` at runtime. Now, we can minimize the `NullReferenceException` at runtime with the new Null Reference Type which was introduced in C# 8.0.

Note: Most programmers assumed that the Reference Type is meant to accept both null and non-null. There was not any explicit handling required and unfortunately, it is one of the reasons for `NullReferenceException`. In C# 8.0, nullable reference types and non-nullable reference types are introduced which allows us to decide whether we need to make the reference variable nullable or non-nullable. If this is not clear at the moment, then don't worry, we will explain this with multiple examples.

Example to Understand Nullable Reference Types in C# 8:

Nullable Reference Types aren't checked to ensure they aren't assigned or initialized to null. However, the compiler uses flow analysis to ensure that any variable of a nullable reference type is checked against null before it's accessed or assigned to a non-nullable reference type.

C# 8.0 introduces Nullable Reference Types. This feature is another way to specify whether a given parameter, variable, or return value can be null or not. In C# 8, the compiler emits a warning or error if a variable that must not be null is assigned to null. Those warnings can help you to find and fix most of your `NullReferenceException` before they blow up at runtime.

Example using C# 7 or Older Version

Let us understand this with an example. Please have a look at the below example. The following example I have written using Visual Studio 2017 and using C# 7. In the below example, you will not get any warning.

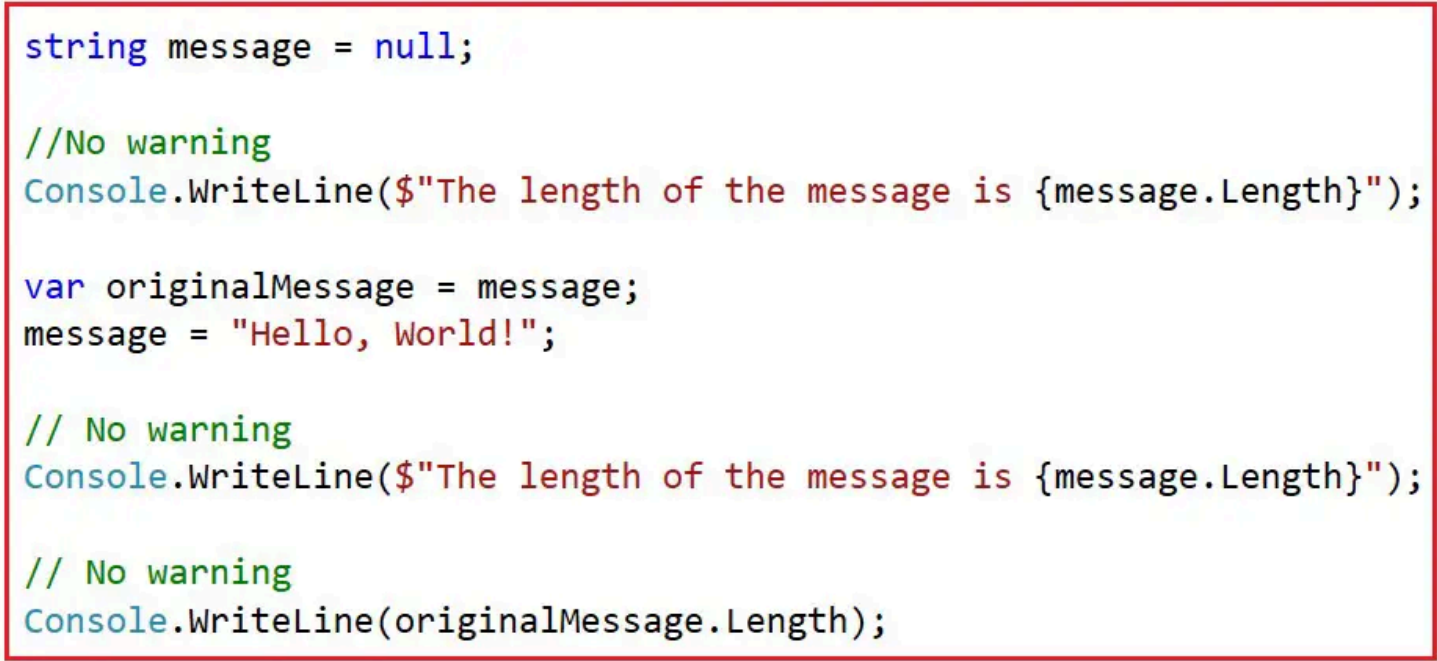
```
using System;
namespace NullableReferenceTypesDemo
{
```

```

public class Program
{
    public static void Main()
    {
        string message = null;
        //No warning
        Console.WriteLine($"The length of the message is {message.Length}");
        var originalMessage = message;
        message = "Hello, World!";
        // No warning
        Console.WriteLine($"The length of the message is {message.Length}");
        // No warning
        Console.WriteLine(originalMessage.Length);
    }
}

```

The following image shows code in Visual Studio 2017 with C# 7 without warning.



```

string message = null;

//No warning
Console.WriteLine($"The length of the message is {message.Length}");

var originalMessage = message;
message = "Hello, World!";

// No warning
Console.WriteLine($"The length of the message is {message.Length}");

// No warning
Console.WriteLine(originalMessage.Length);

```

So, to show a warning we need to enable nullable reference types.

How to Enable Nullable Reference Types in C#?

Null-State analysis and variable annotations are disabled by default for existing projects meaning that all reference types continue to be nullable. Starting with .NET 6, they're enabled by default for new projects. If you are working with an older version of the .NET Core Application, then you need to enable Nullable annotation. In order to enable Nullable annotations in .NET Core Application, you need to edit the project **.csproj** file and add **<Nullable>enable</Nullable>** in the property group as shown in the below image.

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net6.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Example to Understand Nullable Reference Types in C# 8

So, create a new console application using Visual Studio 2022 and .NET 6 and then copy and paste the following code into the Program class. This is the same code that we used in our previous example with Visual Studio 2017 and C# 7.

```
using System;
namespace Csharp8Features
{
    public class NullableReferenceTypes
    {
        public static void Main()
        {
            //As you are creating a variable to accept null, you need to use ;
            //else you will get warning
            string message = null;
            // warning: Dereference Null.
            // Dereference means accessing the Property of message object
            // As we are accessing for the first time, as it is null, so we are
            an warning
            Console.WriteLine($"The length of the message is {message.Length}")
            //No warning: Accessing Length Property Second Time.
            Console.WriteLine($"The length of the message is {message.Length}")
            //Assigning message property to originalMessage
            //Third time we are accessing the message property, so compiler assumes
            //the message may be null
            var originalMessage = message;
            //Assigning value to message property
            message = "Hello, World!";
            // No warning: Compiler Determined "message" is not null.
            Console.WriteLine($"The length of the message is {message.Length}")
            // No Warning: Compiler assumes originalMessage is may be null
            Console.WriteLine(originalMessage.Length);
        }
    }
}
```

At the end of this article, you will understand how this feature work to produce warnings when our code dereferences a null value. **Dereferencing a variable means accessing one of its members using the dot (.) operator**. For a better understanding, please have a look at the below code.

```
string message = "Welcome to C#.NET";  
int length = message.Length; // dereferencing "message"
```

Note: When you dereference a variable whose value is null, then you will get a `NullReferenceException` at runtime.

So, with this `Nullable` Annotation, you can address the warnings in your code which ultimately reduces the chances of runtime `NullReferenceException`. The compiler will determine the null state of a variable. A variable is either not-null or may be null. The compiler determines that a variable is **not-null** in two ways:

1. The variable has been assigned a value that is known to be not null.
2. The variable has been checked against null and hasn't been modified since that check.

Any variable that the compiler hasn't determined as **not null** is considered **may be null**. The analysis provides warnings in situations where you may **accidentally dereference a null value** (when you access it for the first time). The compiler produces warnings based on the null state.

1. When a variable is not-null, that variable may be dereferenced safely.
2. When a variable is maybe-null, that variable must be checked to ensure that it isn't null before dereferencing it.

For a better understanding, please have a look at the below code. As you can see when you access the `Length` Property of the `message` object for the first time, it is giving you one warning. This is because the compiler assumes that you are doing this accidentally and hence give you one chance to correct it. But when you access the `Length` Property of the `message` object for the second time, you are not getting any warning. This is because you are not doing this accidentally, but you are accessing it intentionally.

```

//As you are creating a variable to accept null, you need to use ?
//else you will get warning
string message = null;

// warning: Dereference Null.
Console.WriteLine($"The length of the message is {message.Length}");

//No warning: Accessing Length Property Second Time.
Console.WriteLine($"The length of the message is {message.Length}");

var originalMessage = message;
message = "Hello, World!";

// No warning: Compiler Determined "message" is not null.
Console.WriteLine($"The length of the message is {message.Length}");

// No Warning: Compiler assume originalMessage is may be null
Console.WriteLine(originalMessage.Length);

```

Nullable state analysis and the warnings which are given by the compiler will help you to avoid runtime Null Reference Exception. So, when you run the above code, you will get a runtime error as shown in the below image.

```

// warning: Dereference Null.
Console.WriteLine($"The length of the message is {message.Length}");

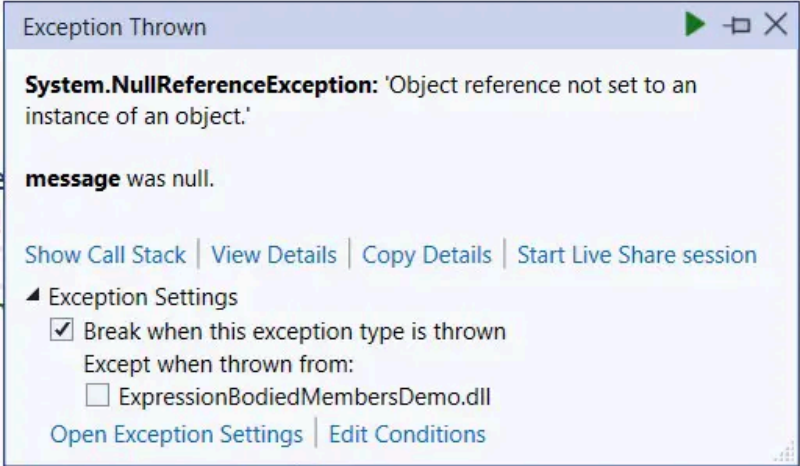
//No warning: Accessing
Console.WriteLine($"The

var originalMessage = me
message = "Hello, World!

// No warning: Compiler
Console.WriteLine($"The

// No Warning: Compiler
Console.WriteLine(originalMessage.Length);

```



In the above example, you are also getting a warning while accessing the Length Property for the first time using the message object. And this will help you to avoid the above runtime Null Reference Exception. In the below example, we have handled the null exception.

```

using System;
namespace Csharp8Features
{
    public class NullableReferenceTypes
    {
        public static void Main()
        {
            string message = null;
            // No warning: Null Checking
            if (!string.IsNullOrEmpty(message))
            {
                Console.WriteLine($"The length of the message is {message.Length}");
            }
            var originalMessage = message;
            message = "Hello, World!";
            // No warning. Analysis determined "message" is not null.
            Console.WriteLine($"The length of the message is {message.Length}");
            // warning! Analysis determined "originalMessage" is null.
            Console.WriteLine(originalMessage.Length);
        }
    }
}

```

So, this new feature of C# 8 helps us to solve the `NullReferenceException`

Enable Nullable Annotations in a File or Part of the Code

You can put the **#nullable enable** directive where you want to enable the functionality and the **#nullable disable** directive, where you want to disable the functionality. If you put **#nullable disable** on the file head, that will not allow the nullability check for the whole file. For a better understanding, please have a look at the below image.

```

public static void Main()
{
    #nullable disable
    string message = null;

    // warning: dereference null.
    Console.WriteLine($"The length of the message is {message.Length}");

    var originalMessage = message;
    message = "Hello, World!";

    // No warning. Analysis determined "message" is not null.
    Console.WriteLine($"The length of the message is {message.Length}");

    // warning!
    Console.WriteLine(originalMessage.Length);
}

```

If you put **#nullable enable** on the file head, that should allow the nullability check for the whole file. For a better understanding, please have a look at the below image.

```

public class NullableReferenceTypes
{
    0 references
    public static void Main()
    {
        #nullable enable
        string message = null;

        // warning: dereference null.
        Console.WriteLine($"The length of the message is {message.Length}");

        var originalMessage = message;
        message = "Hello, World!";

        // No warning. Analysis determined "message" is not null.
        Console.WriteLine($"The length of the message is {message.Length}");

        // warning!
        Console.WriteLine(originalMessage.Length);
    }
}

```

Finally, you can restore the default setting as below:

#nullable restore

Let us see a few more examples to get more clarity.

Example 1:

When we use the question mark '?' when declaring the variable, then that variable is a nullable variable and can accept a null value. So, in the below example, the first statement will not give any warning but while accessing the Length property, it is giving one warning, so you need to take this.

```
using System;
namespace Csharp8Features
{
    public class NullableReferenceTypes
    {
        public static void Main()
        {
            // Is Ok, nullableString it can be null and it is null.
            string? nullableString = null;
            // WARNING: nullableString is null! Take care!
            Console.WriteLine(nullableString.Length);
        }
    }
}
```

Example 2:

When you create a property of nonnull type without using the question mark, and if you are not initializing that property using the constructor, then you will get a warning. But as soon as you are initializing that property using the constructor, the warning will be gone which is shown in the below example.

```
using System;
namespace Csharp8Features
{
    class Person
    {
        // Warning Name is Non Null!
        public string Name { get; set; }
        // No Warning Name is null!
        public string? NullableName { get; set; }
        //Enable the below code then the warning above will be disappeared
        //public Person(string name)
        //{
        //    Name = name;
        //}
    }
}
```

Note: The first property i.e. Name is a reference type, and it is null and for this reason the compiler warns you. The Second property i.e. NullableName is a nullable reference type and that's why the compiler is not warning because the NullableName can be null, you have defined it as nullable.

Benefits of Nullable Reference Types in C#

The introduction of this feature from C# 8.0 allows for several benefits that are not present in earlier versions:

1. Allows the programmer to clearly show their intent when declaring variables.
2. Provides protection against Null Reference Exceptions.
3. The compiler warns you if you dereference a nullable reference when it may be null.

Rules for Non-nullable Reference Type in C#

When a variable is not supposed to be null, the compiler enforces some rules to make sure that it is safe to dereference that variable without checking that it is not null.

1. The variable must be initialized to a non-null value.
2. The variable can never be assigned the null value.

Rules for Nullable Reference Type in C#

When a variable can be null, in that case, the compiler enforces different rules to make sure that you have correctly checked for a null reference.

1. The variable may only be dereferenced when the compiler can guarantee that the value is not null.
2. It may be initialized with the default null value and may be assigned the value null in another code.

In the next article, I am going to discuss [Asynchronous Streams in C# 8](#) with Examples. Here, in this article, I try to explain **Nullable Reference Types in C# 8** with Examples. I hope you enjoy this Nullable Reference Types in C# with Examples article.